SOFTWARE MAINTENANCE PORTOFOLIO

Lab 1 (ChangeReqLab):

Feature: Change the canvas orientation from landscape to portrait.

User Story: As a user, I want to change the canvas orientation from landscape to portrait so that I can create drawings that fit portrait documents.

Lab 2 (CLLab):

Domain Class	Responsibility
AttributeKeys	Defines a put of well known attributes.
DefaultDrawing	A default implementation of {@link Drawing} useful for drawing which contain only a few figures
DefaultDrawingView	A default implementation of {@link DrawingView} suited for viewing drawings with a small number of figures
QuadTreeDrawing	An implementation of {@link Drawing} which uses a {@link org.jhotdraw.geom.QuadTree} to provide a good responsiveness for drawings which contain many figures
CanvasToolBar	The toolbar UI component which provides buttons/actions related to canvas operations
CanvasTool	Tool used to interact with the canvas, handling mouse events and enabling user actions on the canvas
ViewToolBar	Toolbar for general view operations

Lab 3 (AnalysisLab1):

Package Name	# of classes	Comments
org.jhotdraw.draw	5	Contains the core drawing framework and it is the primary implementation location

org.jhotdraw.draw.tool	1	Contains interaction tools for manipulating the drawing and only relevant if user changes orientation interactively

org.jhotdraw.draw.action	2	Contains UI components for view controls and used to add a portrait or landscape toggle

Lab 4 (RefactLab)

In the validateHandles method, of the DefaultDrawingView class, there are nested loops, multiple conditionals, and a while loop with continue. This introduces cognitive complexity and the method difficult to understand and maintain. According to [Ker05], this would be Conditional Complexity. What I plan to change is to extract nested logic into smaller methods and reduce branching inside the main method. Currently, the method handles state validation, build handling, retry logic, etc. and decreases readability and maintainability. My strategy is to split the responsibility into multiple smaller helper methods. Choosing the “Replace Conditional Calculations with Strategy” refactoring allows me to separate conditional logic into smaller methods which improves readability and reduces complexity, making future modifications easier.

Lab 5 [ActLab]

Clean Architecture organizes software into layers and its structure isolates business logic from UI and infrastructure. Changes in one layer cause minimal impact on others. In the JHotDraw case study, the feature modifies canvas orientation. This is a presentation concern. Clean Architecture helps place this change in the correct layer.

Single Responsibility was applied by keeping orientation logic inside DefaultDrawingView. Open Closed Principle was respected by extending behavior without modifying drawing classes. Liskov Substitution was preserved by ensuring QuadTreeDrawingView behaves consistently. Interface Segregation Principle was respected by having CanvasTool interact with DrawingView and not needing any orientation specific methods. Dependency Inversion Principle was respected by having UI components such as CanvasToolBar depend on Drawing View and not depend on DefaultDrawingView directly.

Lab 7 [TestLab1]

Unit tests were implemented for core canvas sizing logic. The tests focused on methods responsible for view dimensions and canvas bounds. A best case scenario verified that the preferred size returned the expected dimensions after configuration. Boundary cases tested

zero size canvas bounds and minimal dimension updates through setBounds. Dependencies on Swing rendering were avoided to ensure isolated unit testing. The feature was verified by confirming correct dimension handling, which is essential for supporting portrait and landscape canvas formats.

Lab 9 [TestLab2]

As a user, I want to change the canvas from landscape to portrait so that I can create drawings in portrait format.

BDD Scenarios

Scenario 1, change canvas to portrait

Given a drawing view with landscape orientation
When the user selects portrait format
Then the canvas height becomes greater than width

Scenario 2, keep portrait when already portrait

Given a drawing view already in portrait
When the user selects portrait format
Then the canvas size remains unchanged

Scenario 3, switch back to landscape

Given a drawing view in portrait
When the user selects landscape format
Then the canvas width becomes greater than height